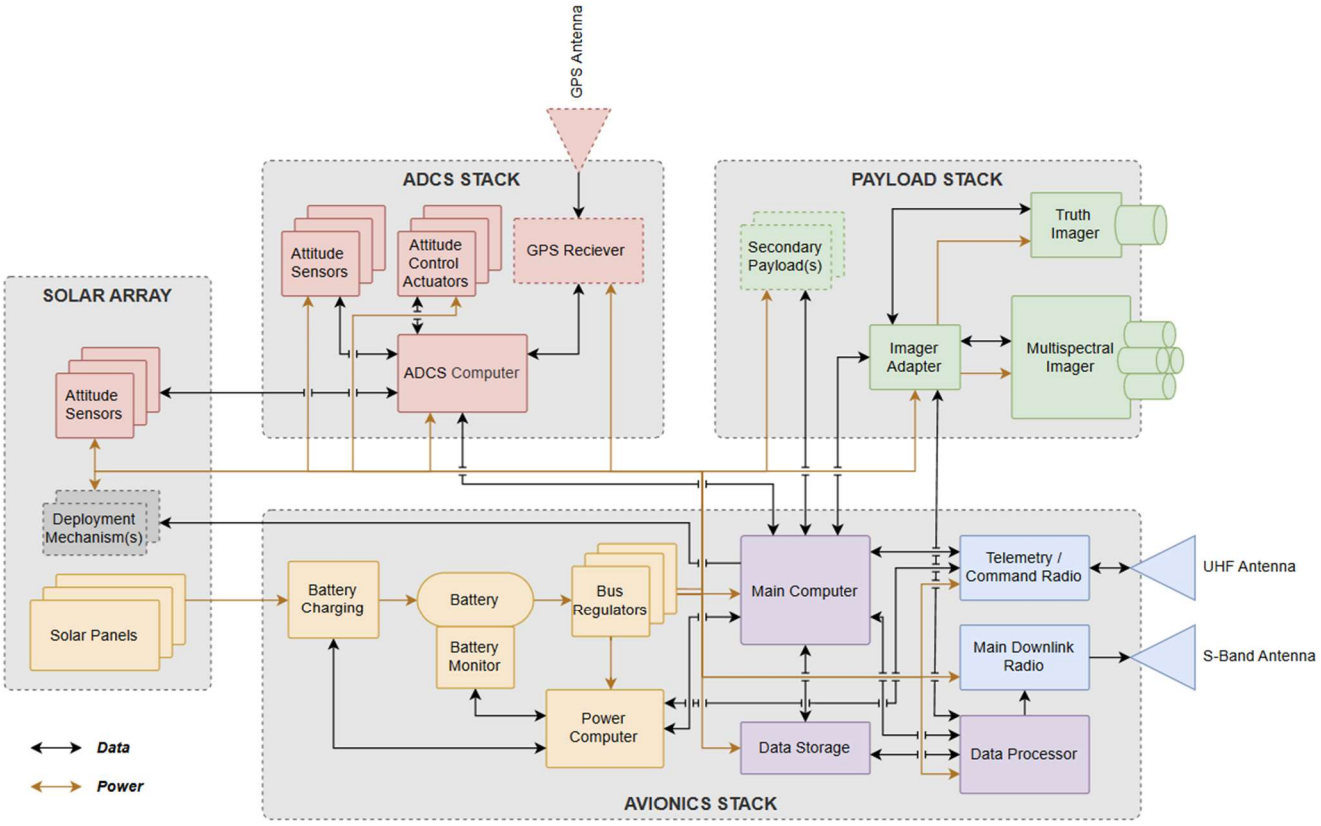
# Assignment 5

Caden O'Leary, Elmoataz Abdellah, Matthew Goldsberry, Seunghun Lee, Kristen Griessman

## Block Diagrams

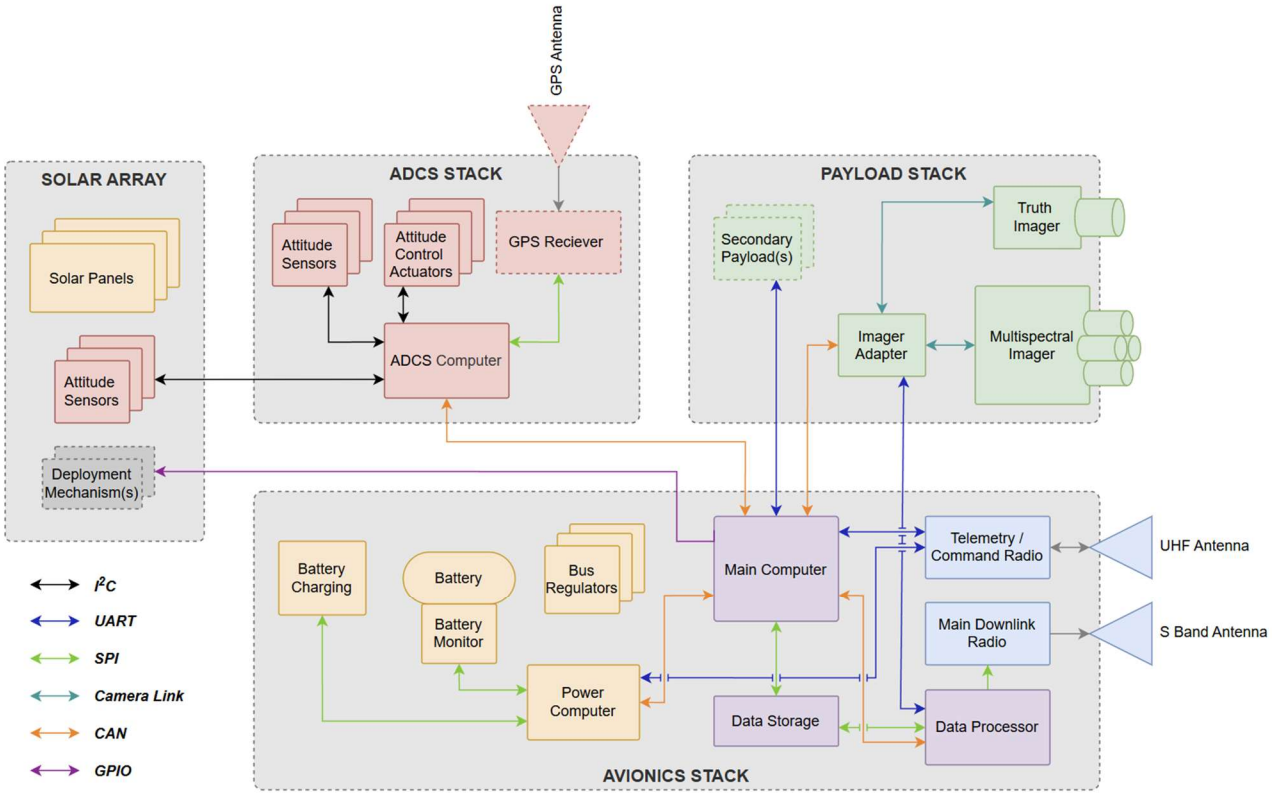
### HABSat-1 Data and Power Lines




**HABSat-1 Data & Power Lines**

Defines the power and dataflow between components in HABSat-1.

# HABSat-1 Protocol Lines




**HABSat-1 Protocol Lines**

Defines the hardware communication protocols between components in HABSat-1.

# Component Responsibility Table

## ADCS Stack

| Component | Responsibility | In | Out | Owner |
|---|---|---|---|---|
| Attitude Sensors | Provide sensor readings to determine the satellite's orientation and the location of the Sun and Earth. | ADCS Computer | ADCS Computer | ADCS Subsystem |
| Attitude Control Actuators | Reaction wheels that spin to control satellite orientation. | ADCS Computer | ADCS Computer | ADCS Subsystem |
| GPS Receiver | Reads antenna data to determine location. | ADCS Computer | ADCS Computer | ADCS Subsystem |
| GPS Antenna | Receives GPS data. | Sensor Data | GPS Antenna | ADCS Subsystem |
| ADCS Computer | Reads sensor data and TLEs to determine future passes, orient the satellite to target, orient the solar panels towards the sun, orient the satellite towards the ground station for downlink, and detumble the satellite upon launch. | Attitude Sensor<br>GPS Receiver | Attitude Sensors (Solar Array, ADCS Stack)<br>Attitude Sensor Actuator | ADCS Subsystem **Seunghun** |

## Payload Stack

| Component | Responsibility | In | Out | Owner |
|---|---|---|---|---|
| Secondary Payload | TBD. Working to find a partner for this. | Main Computer | Main Computer | Payload Subsystem **Caden** |
| Imager Adapter | Controls the truth and multispectral imager. | Main Computer | Main Computer<br>Multispectral Imager<br>Truth Imager | Payload Subsystem **Kristen** |
| Multispectral Imager | Captures images matching the color wavelengths of light for algae. | Multispectral Imager | Image Adapter | Payload Subsystem |
| Truth Imager | Captures the real image of the lake to match multispectral image data. | True images | Image Adapter | Payload Subsystem |

## Avionics Stack

| Component | Responsibility | In | Out | Owner |
|---|---|---|---|---|
| Main Computer | Orchestrate mission operations based on satellite state by commanding other components to complete tasks. | ACDS<br>Computer<br>Secondary Payload<br>Imager Adapter<br>Telemetry / Command Radio<br>Data Processors<br>Data Storage<br>Power<br>Computer | ACDS<br>Computer<br>Secondary Payload<br>Imager Adapter<br>Telemetry / Command Radio<br>Data Processors<br>Data Storage<br>Power<br>Computer | **Matthew**<br>**Caden**<br>**Elmoataz**<br>**Kristen**<br>**Seunghun** |
| Data Storage | Storage of payload and mission software | Data Processor<br>Main Computer | Data Processor<br>Main Computer | **Elmoataz** |

| | | | | |
|---|---|---|---|---|
| Data Processor | Load images and perform chlorophyll index conversion on multispectral images. | | | **Matthew** |
| Main Downlink Radio | Downlink of multispectral images | S-Band Antenna | Data Processor | Comms Subsystem **Caden** |
| S-Band Antenna | Transmit image data via S-band. | S-band Signals | Main Downlink Radio | Comms Subsystem |
| Telemetry / Command Radio | Uplink commands from ground station and downlink | UHF Antenna | Main Computer | Comms Subsystem **Caden** |
| UHF Antenna | Transmit and receive UHF signals | UHF Signals | Telemetry / Command Radio | Comms Subsystem |
| Power Computer | Monitors power state and ensure the satellite stays within safe thresholds, while also providing insight to the Main Computer | Battery Charging Battery Monitor Main Computer Telemetry / Command Radio | Main Computer, Battery Charging, Telemetry / Command Radio | Power Subsystem **Seunghun** |
| Battery Charging | Regulate charging current from solar panels to safely charge the satellite battery. | Solar Panels, Power Computer | Battery, Power Computer | Power Subsystem |
| Battery | Store energy generated by solar panels and supply | Battery Charging | Bus Regulators, Battery Monitor | Power Subsystem |

| | | | | |
|---|---|---|---|---|
| | direct power to the power bus. | | | |
| Battery Monitor | Measure real time battery voltage, current, and temperature to report state of charge. | Battery | Power Computer | Power Subsystem |
| Bus Regulators | Step down and stabilize battery voltage to deliver regulated power lines to avionics components. | Battery | Main Computer, Power Computer, Subsystems | Power Subsystem |

## Solar Array

| Component | Responsibility | In | Out | Owner |
|---|---|---|---|---|
| Solar Panels | Power batteries through sunlight | Solar Panels | Battery Charging | Power Subsystem |
| Attitude Sensors | Determine orientation of the satellite | ACDS Computer | ACDS Computer | ACDS Subsystem |
| Deployment Mechanisms | Deploy the solar array | Main Computer | Deployment Mechanisms | Structures Subsystem **Caden** |

## Interface Specification Table

### From Main Computer:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| ADCS Computer | packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | 12 bytes<br>{<br>CAN STR: 0x00<br>CAN ID: 0x00<br>CAN Payload 0x00000000<br>CAN CRC: 0x00<br>CAN ETX: 0x00<br>} |
| Power Computer | Packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | 12 bytes<br>{<br>CAN STR: 0x00<br>CAN ID: 0x00<br>CAN Payload 0x00000000<br>CAN CRC: 0x00<br>CAN ETX: 0x00<br>} |
| Image Adapter | Packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | 12 bytes<br>{<br>CAN STR: 0x00<br>CAN ID: 0x00<br>CAN Payload 0x00000000<br>CAN CRC: 0x00<br>CAN ETX: 0x00<br>} |
| Deployment Mechanisms | Signal(H/L) | GPIO | Retry | 0 is High to Low<br>1 is Low to High<br>{PIN ID:X DATA:0}<br>{PIN ID:X DATA:1} |
| Secondary Payload | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | 10+N byte (fixed)<br>{<br>Preamble: 0x00<br>NodeID: 0x00<br>FramID: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>ETX: 0x00<br>} |
| Telemetry / Command Radio | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | 10+N byte (fixed)<br>{<br>Preamble: 0x00<br>NodeID: 0x00<br>FramID: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>ETX: 0x00<br>} |

| | | | | |
|---|---|---|---|---|
| Data Storage | I/O | SPI | Retry<br>Device Reset | N/A |
| Data Processor | Packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | CAN ID: 0x10<br>CAN Payload 0x00000000 |

## From Deployment Mechanisms:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Main Computer | Signal(H/L) | GPIO | Retry | 0 is High to Low<br>1 is Low to High<br>{PIN ID:X DATA:0}<br>{PIN ID:X DATA:1} |

## From Attitude Sensors:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| ADCS Computer | Frame | I2C | N/A (handled by subsystem team) | N/A (handled by subsystem team) |

## From Attitude Control Actuators:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| ADCS Computer | Frame | I2C | N/A (handled by subsystem team) | N/A (handled by subsystem team) |

## From GPS Receiver:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| ADCS Computer | I/O | SPI | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| GPS Antenna | I/O | SPI | N/A (handled by hardware) | N/A (handled by hardware) |

## From ADCS Computer:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Attitude Sensor | Frame | I2C | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| Attitude Control Actuator | Frame | I2C | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| GPS Receiver | I/O | SPI | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| Main Computer | Packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | 12 bytes<br>{<br>CAN STR: 0x00<br>CAN ID: 0x00<br>CAN Payload 0x00000000<br>CAN CRC: 0x00<br>CAN ETX: 0x00<br>} |

## From Secondary Payload:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Main Computer | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | 10+N byte (fixed)<br>{<br>Preamble: 0x00<br>NodeID: 0x00<br>FramID: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>ETX: 0x00<br>} |

## From Imager Adapter:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Main Computer | Frame | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | 12 bytes<br>{<br>CAN STR: 0x00<br>CAN ID: 0x00<br>CAN Payload 0x00000000<br>CAN CRC: 0x00<br>CAN ETX: 0x00<br>} |
| Truth Imager | Direct | Camera Link | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| Multispectral Imager | Direct | Camera Link | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| Data Processor | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | N byte (dynamic)<br>{<br>Preamble: 0x00<br>ID: 0x00<br>Length1: 0x00<br>Length2: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>} |

## From Truth Imager:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Imager Adapter | Direct | Camera Link | N/A (handled by subsystem team) | N/A (handled by subsystem team) |

## From Multispectral Imager:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Imager Adapter | Direct | Camera Link | N/A (handled by subsystem team) | N/A (handled by subsystem team) |

## From Battery Charging:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Power Computer | I/O | SPI | N/A (handled by subsystem team) | N/A (handled by subsystem team) |

## From Battery Monitor

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Power Computer | I/O | SPI | N/A (handled by subsystem team) | N/A (handled by subsystem team) |

## From Power Computer

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Battery Charging | I/O | SPI | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| Battery Monitor | I/O | SPI | N/A (handled by subsystem team) | N/A (handled by subsystem team) |
| Main Computer | Packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | 12 bytes<br>{<br>CAN STR: 0x00<br>CAN ID: 0x00<br>CAN Payload 0x00000000<br>CAN CRC: 0x00<br>CAN ETX: 0x00<br>} |

## From Data Storage:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Main Computer | I/O | SPI | Retry<br>Device Reset | {<br>SS: 0x00<br>SCLK: 0x00<br>MOSI: 0x00<br>MISO: 0x00<br>} |
| Data Processor | I/O | SPI | Retry<br>Device Reset | {<br>SS: 0x00<br>SCLK: 0x00<br>MOSI: 0x00<br>MISO: 0x00<br>} |

## From Data Processor:

| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Main Computer | Packet | CAN BUS | CRC Check<br>Bit stuffing<br>Frame Check<br>Retry | CAN ID: 0x10<br>CAN Payload 0x00000000 |
| Data Storage | I/O | SPI | Retry<br>Device Reset | {<br>SS: 0x00<br>SCLK: 0x00<br>MOSI: 0x00<br>MISO: 0x00<br>} |
| Imager Adapter | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | N byte (dynamic)<br>{<br>Preamble: 0x00<br>ID: 0x00<br>Length1: 0x00<br>Length2: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>} |
| Main Downlink Radio | I/O | SPI | Retry<br>Device Reset | {<br>SS: 0x00<br>SCLK: 0x00<br>MOSI: 0x00<br>MISO: 0x00<br>} |

## From Telemetry/ Command Radio

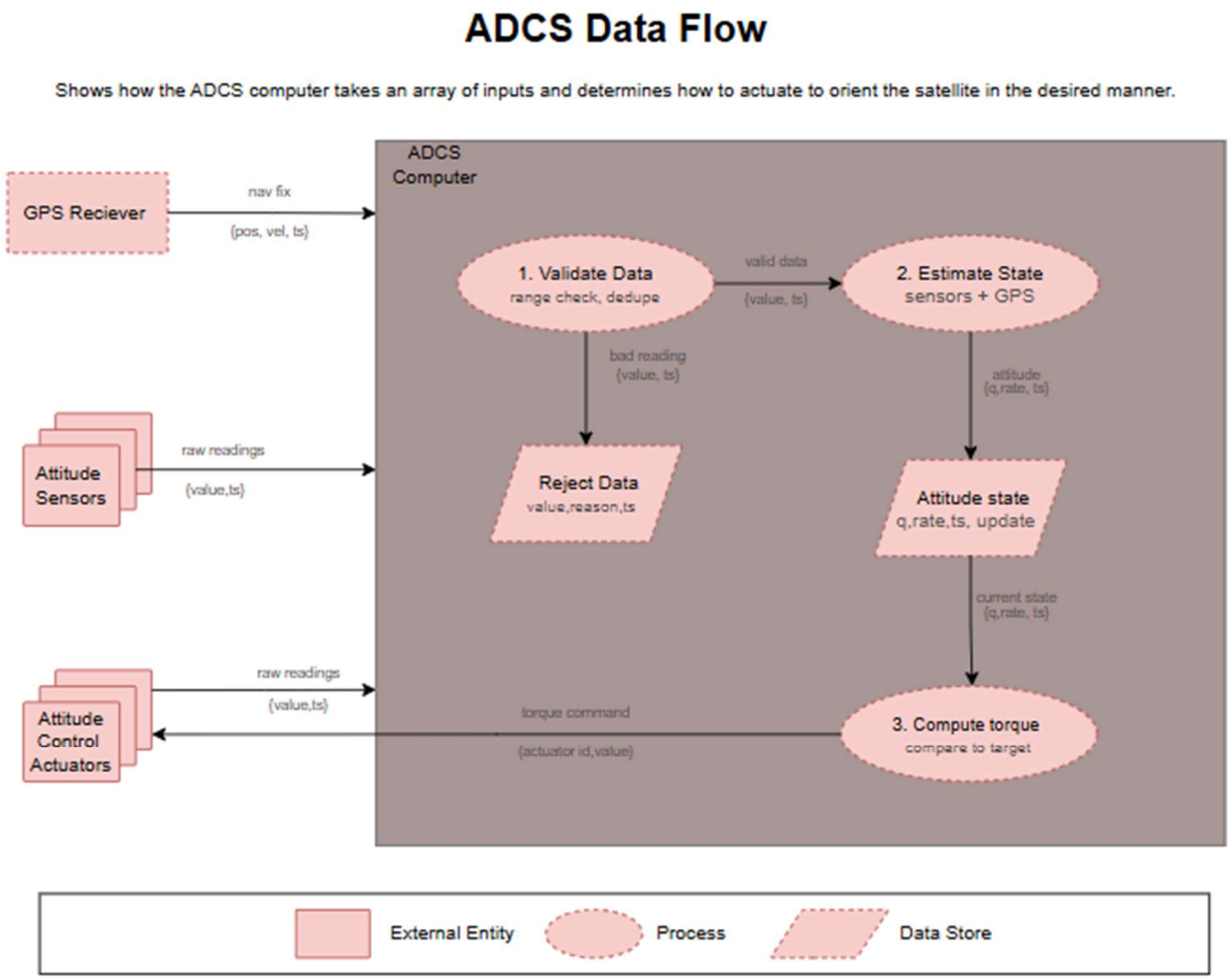
| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Main Computer | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | 10+N byte (fixed)<br>{<br>Preamble: 0x00<br>NodeID: 0x00<br>FramID: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>ETX: 0x00<br>} |
| Power Computer | Frame | UART | Parity Error<br>CRC Check<br>ACK/NACK<br>Retry | 10+N byte (fixed)<br>{<br>Preamble: 0x00<br>NodeID: 0x00<br>FramID: 0x00<br>Payload: dynamic 0x0...<br>Checksum: 0x00<br>ETX: 0x00<br>} |
| UHF Antenna | Direct | Direct | N/A (handled by hardware) | N/A (handled by hardware) |

## From Main Downlink Radio

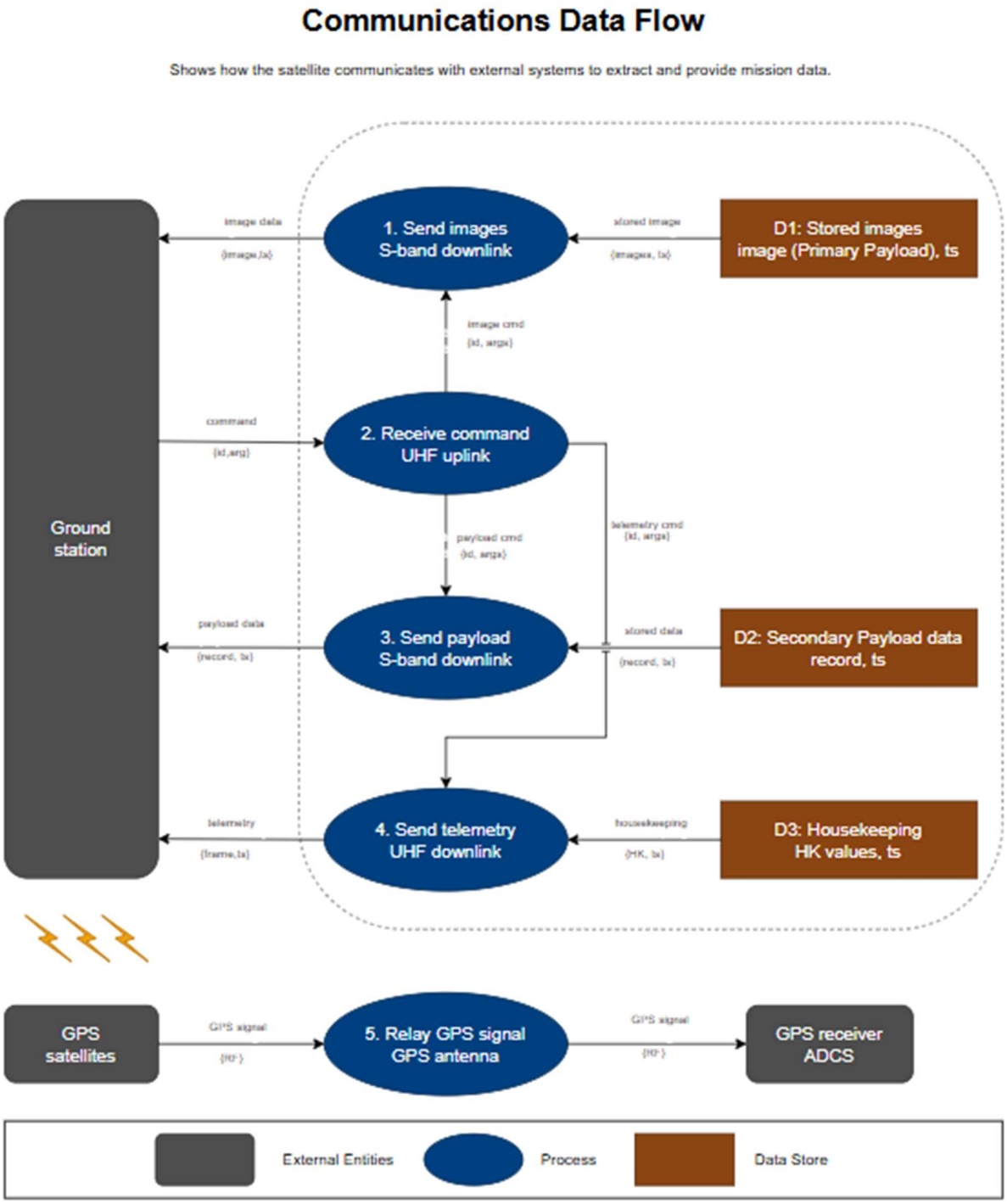
| Interface | Format | Protocol | Error Handling | Examples |
|---|---|---|---|---|
| Data Processor | I/O | SPI | Device Reset | {<br>SS: 0x00<br>SCLK: 0x00<br>MOSI: 0x00<br>MISO: 0x00<br>} |
| S-band Antenna | Direct | Direct | N/A (handled by hardware) | N/A (handled by hardware) |

# Data Flow Diagrams

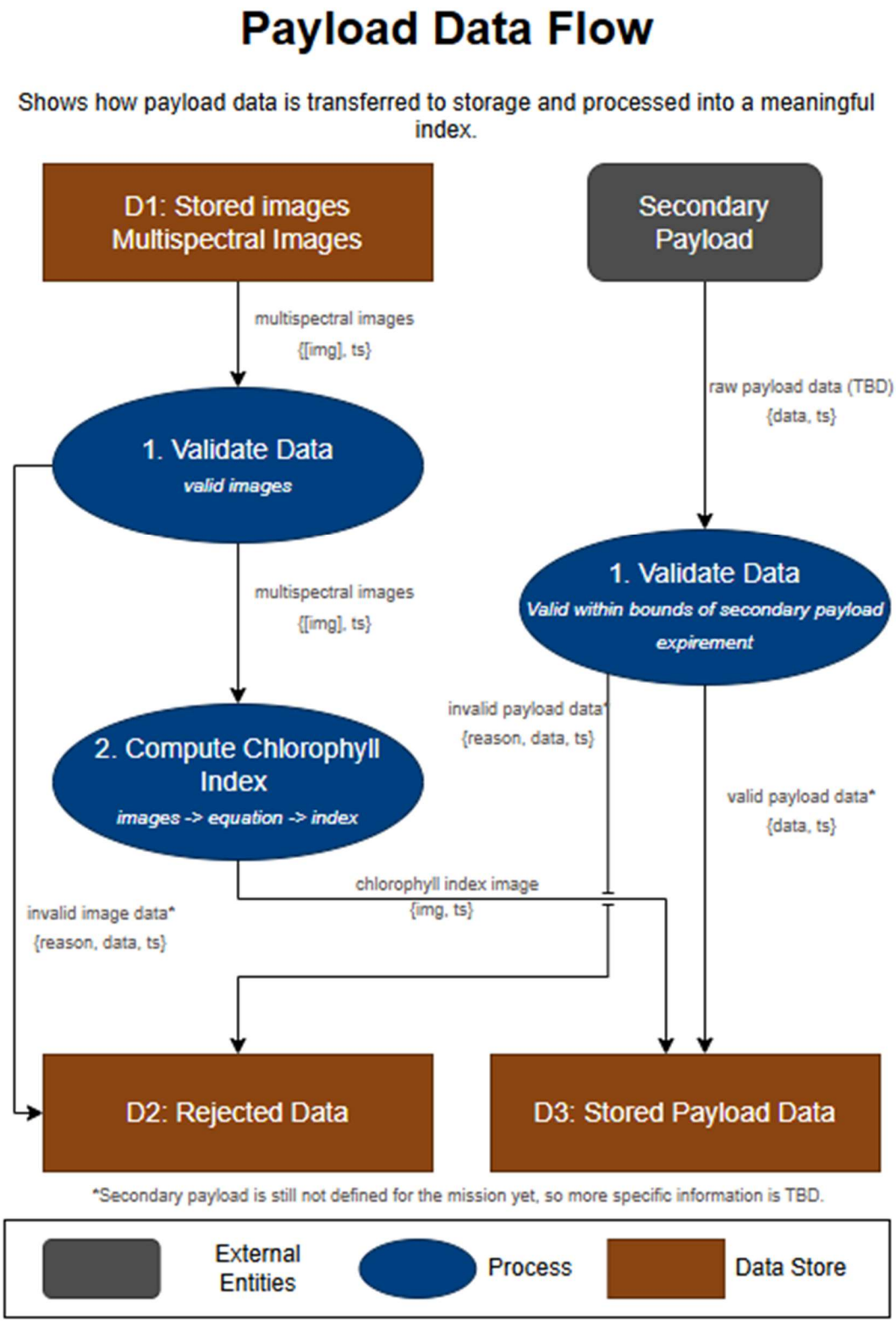
## ADCS Data Flow



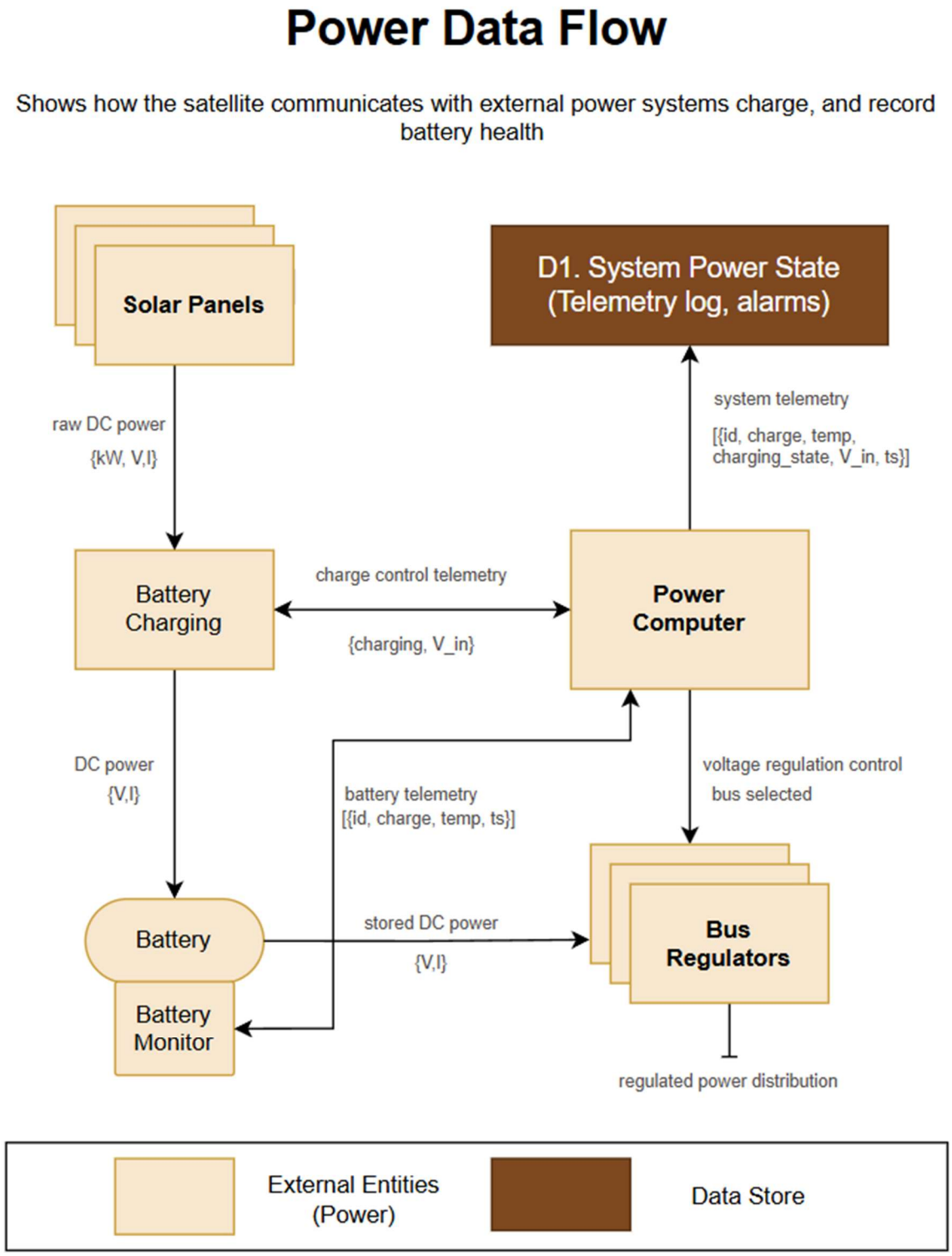
## Communications Data Flow



## Payload Data Flow



## Power Data Flow



## Architecture Pattern

For the HABSat-1 On-Board Computer (OBC) flight software, we use a combination of the Embedded Sensor-Actuator pattern, complemented by a Client-Server model for ground station links.

### Embedded Sensor-Actuator Pattern

Governs the low-level hardware interaction layers, managing telemetry acquisition and actuator commands across satellite subsystems (e.g., EPS, ADCS, payloads).

### Client-Server Pattern

Governs the communication interface and command/telemetry exchange between the OBC and the ground station.

### Fit to problem shape

The data and control in a satellite move hierarchically and cyclically—subsystem telemetry flows upward, CDH application logic processes commands, and control signals flow back down to actuators. The **Embedded Sensor-Actuator** and **Layered** patterns directly match this unidirectional and hierarchical flow of control and data between hardware devices and mission tasks.

### Team skillset

As a student-led team (CubeCats), the architecture must be realistic to build, debug, and maintain. Layered and embedded sensor-actuator designs are standard paradigms taught in embedded systems courses, ensuring the team has the practical competency to implement them without getting bogged down by overly complex, esoteric frameworks.

### Performance & timing needs

Spacecraft flight software must meet strict real-time deadlines and execute scheduled mission tasks deterministically. A **Layered** design ensures clear, bounded execution pathways with minimal dynamic overhead, keeping latency predictable and preventing priority inversions during critical flight states.

### Scalability needs

HABSat-1 may need to accommodate future mission phases, new payload data, and additional telemetry channels. The **Layered** pattern handles this scalability gracefully by allowing developers to plug in new application modules or expand CDH capabilities at the top layer without rewriting lower-level hardware drivers.

## Hardware constraints

CubeSat On-Board Computers operate under tight hardware limits (low processing power, constrained memory, and strict power budgets). This pattern combination avoids high memory footprints, dynamic allocations, and the processing overhead of distributed architecture, fitting cleanly onto embedded microcontrollers or single-board computers.

## Rejected Pattern

Microservices

## Reason for Rejection

Microservices were rejected because they introduce excessive network overhead, inter-process communication complexity, and memory footprint requirements. Running independent microservices across containerized or networked nodes exceeds the constrained processing and resource limits of a single-board CubeSat OBC.

## Decision Log

Architecture Decision Records (ADRs), covering the project design choices for HABSat-1:

### ADR-001: Addition of a Dedicated Data Processor

- **Title:** ADR-001: Addition of a Dedicated Data Processor
- **Status:** Accepted, Week 5 (9/30/2026). Revisit if payload data throughput requirements decrease.
- **Context:** High-bandwidth payload tasks (such as image capture and processing) threaten to saturate the CPU and memory resources of the primary On-Board Computer (OBC).
- **Options considered:** A. Process all data and telemetry on the single main OBC, B. Add a dedicated secondary data processor/coprocessor, C. Offload all raw processing completely to the ground station.
- **Decision:** Option B. Offloads compute-heavy payload tasks, protecting core Command and Data Handling (CDH) reliability and real-time scheduling.
- **Consequences:** Adds hardware mass, power drawing, and inter-processor communication complexity.
- **Owner and reviewers:** CubeCats OBC Team; reviewed by senior chief engineer on September 30th.

### ADR-002: Route S-Band Downlink Connection Directly to Data Processor

- **Title:** ADR-002: Route S-band downlink connection to data processor
- **Status:** Accepted, Week 5 (9/30/2026). Revisit if primary OBC bandwidth bottlenecks disappear.
- **Context:** High-rate image files and science data compiled by the newly added data processor need efficient transmission paths to the S-band radio without bottlenecking the main OBC.
- **Options considered:** A. Route all payload files through the main compute before downlink, B. Connect the S-band downlink directly to the data processor, C. Store data entirely on external flash until ground command retrieval.
- **Decision:** Option B. Bypasses main computer bottlenecks for high-rate data transmission, maximizing downlink efficiency.

- **Consequences:** Requires multiplexing or arbitration logic to manage radio access between the main OBC (telemetry/commands) and the data processor (science data).
- **Owner and reviewers:** CubeCats OBC and Comms Team; reviewed by senior chief engineer on September 30th.

## ADR-003: Transition Image Adapter Connection from UART to CAN Bus

- **Title:** ADR-003: Transition image adapter connection from UART to CAN
- **Status:** Accepted, Week 5 (9/30/2026). Revisit if data rate requirements exceed CAN bus limits.
- **Context:** The current point-to-point UART connection linking the image adapter lacks robust multi-drop capabilities, error-checking, and noise immunity for reliable satellite data bus route.
- **Options considered:**
    - A. Keep point-to-point UART.
    - B. Upgrade to high-speed SPI.
    - C. Transition to a Controller Area Network (CAN) bus.
- **Decision:** Option C. Provides robust multi-drop capabilities, superior error handling, and high noise immunity suited for harsh space environments.
- **Consequences:** CAN controller hardware support on endpoints and implementation of higher-layer bus protocol handling.
- **Owner and reviewers:** CubeCats OBC Team; reviewed by senior chief engineer on September 30th.

## ADR-004: Attach Backup Telemetry Radio Connection to Power Computer

- **Title:** ADR-004: Attach telemetry radio connection to power computer for failover
- **Status:** Accepted, Week 5 (9/30/2026). Revisit if main computer reliability testing proves fault-proof.
- **Context:** A catastrophic failure of the main computer would result in total loss of communication and mission awareness unless a failsafe reporting mechanism exists.
- **Options considered:**

    - A. Rely solely on main computer watchdog resets.
    - B. Install a completely independent secondary watchdog board.
    - C. Attach a backup connection from the Power Computer (EPS) to the telemetry radio.
- **Decision:** Option C. Enables critical health beacon transmissions during main computer faults using existing power subsystem hardware.
- **Consequences:** Introduces a cross-subsystem hardware dependency between power and communications; requires failover switching protocol logic.
- **Owner and reviewers:** CubeCats OBC and Power Teams; reviewed by senior chief engineer on September 30th.